

数据结构与算法考试题型与复习范围 (2017)

☞期末考试形式

笔试+闭卷，百分制。

☞总评成绩构成

平时成绩 30% + 期末成绩 70%，

其中，平时成绩包括考勤、纸质作业和实验成绩等。

☞期末考试题型

一. 单选题 (含 20 个小题，每小题 2 分，计 40 分)

二. 简答题 (含 5 个小题，每小题 6 分，计 30 分)

1. 表结构。

2. 树结构。

3. 图结构。

4. 查找。

5. 排序。

三. 设计题 (含 3 个小题，每小题 10 分，计 30 分)

1. 表算法。

2. 树算法。

3. 图算法。

☞期末复习范围

数据结构基本概念

算法及其复杂性

<https://www.cnblogs.com/suiying/p/4748490.html>

线性表；顺序表

链表；循环链表；双向链表

栈；队列；循环队列

括号匹配检验；算术表达式求值

递归函数和递归算法

哈希表基本概念；哈希函数；处理冲突方法

<http://mp.weixin.qq.com/s?biz=MzI1MTIzMzI2MA==&mid=2650562023&idx=2&sn=fedf0075bcce5f27f1afd14026952e67&chksm=f1feeb64c689627209c69c33bb4060e2b3a218df436cea75b03c0c86d8c3b00d0fa5eef7672&mpshare=1&scene=23&srcid=0102wurTuTzdIDORG8Lo0WgW#rd>

树的基本概念；二叉树的性质与存储结构

<http://mp.weixin.qq.com/s?biz=MzI1MTIzMzI2MA==&mid=2650560859&idx=1&sn=37b2168d843282f9366114954346c0eb&chksm=f1feefd8c68966cea139400e03843b3eeab0fbcfcd95476152a428f8ab298fc7decbdb060207&mpshare=1&scene=23&srcid=0102CEth3T6kewnJBmLxHTlK#rd>

二叉树遍历的递归算法和非递归算法

先序后继线索二叉树

哈夫曼树；哈夫曼编码

二叉排序树；平衡二叉树

http://mp.weixin.qq.com/s?__biz=MzI1MTIzMzI2MA==&mid=2650561808&idx=2&sn=7b0f10be87eddfabe01b8b200b24429e&chksm=f1feeb93c68962850d2f8ad59f2f6483550597a97902b8487a80c6f172a339a11cd802c95c76&mpshare=1&scene=23&srcid=0102e3qxt9Ak6Noysp6LroeT#rd

树的存储结构(三种基本存储结构)

图的基本概念与存储结构(邻接矩阵、邻接表)

深度优先搜索遍历；广度优先搜索遍历；简单路径

http://mp.weixin.qq.com/s?__biz=MzI1MTIzMzI2MA==&mid=2650560732&idx=1&sn=20f6bc8d938b041ac6243f9802f3c4f4&chksm=f1feee5fc6896749986d0c7e034d3c27237a9dffdb73647214affdb86d20e94bfb68a78d3ca2b&mpshare=1&scene=23&srcid=0102MHuzCQ2FUUAf0V7ZC2lX#rd

最小生成树；Prim 算法；Kruskal 算法

http://mp.weixin.qq.com/s?__biz=MzI1MTIzMzI2MA==&mid=2650561866&idx=1&sn=ac0fe608294d5588b44f1a6d611c8b44&chksm=f1feebc9c68962dfa472cc66d5d35ac5bc91c36459db9226259e9945c2201f0131df1bfebf5b&mpshare=1&scene=23&srcid=0102mFhtCEhc4oZ3o2xyUsTR#rd

http://mp.weixin.qq.com/s?__biz=MzI1MTIzMzI2MA==&mid=2650560017&idx=1&sn=520c234a34de88bb60dffcf471d6d6718&mpshare=1&scene=23&srcid=0102xctdBprubutMw89w5pAP#rd

最短路径；Dijkstra 算法

http://mp.weixin.qq.com/s?__biz=MzI1MTIzMzI2MA==&mid=2650561859&idx=1&sn=a34e2882574e8d93c6b8c0968bc5be17&chksm=f1feebc0c68962d6907fac86d16718b52fc8ec86489eab76f21dbebdb167f36d179a6d2a89b8&mpshare=1&scene=23&srcid=0102A5cAlWKkTInjeq3dYltp#rd

有向无环图；拓扑排序；关键路径

http://mp.weixin.qq.com/s?__biz=MzI1MTIzMzI2MA==&mid=2650561873&idx=2&sn=936597c060cf874d96a92c0b944329a6&chksm=f1feebd2c68962c4d598dd8471247ca8dce415bbbb557blaff1d85fe0b0d0ef52f2ca742f97f&mpshare=1&scene=23&srcid=0102ynQkrTpI2k60tU3VDKmm#rd

顺序查找；折半查找；

http://mp.weixin.qq.com/s?__biz=MzI1MTIzMzI2MA==&mid=2650560820&idx=1&sn=3fb65b2aacbl1d1e5f6a9dd1a9753591a&chksm=f1feefb7c68966a1ddacd702734dd8b46be0bf4c09c95cd61e4ac7884964755536167a4ab522&mpshare=1&scene=23&srcid=0102fJ2Dek7tC7oAeirfFpjx#rd（七大查找）

http://mp.weixin.qq.com/s?__biz=MzI1MTIzMzI2MA==&mid=2650560135&idx=1&sn=81d2c61e66738f3869ec4e96ff15e38&mpshare=1&scene=23&srcid=0102gJCHeWzw6Juv8FIImNGhX#rd（八大排序）

插入排序；选择排序；起泡排序；

http://mp.weixin.qq.com/s?__biz=MzI1MTIzMzI2MA==&mid=2650561519&idx=2&sn=82817337616668ef129d8f7211fc1ea8&chksm=f1feed6cc689647a9b03f1e116

[9e81ceb42804831e2a628ebf60830c64342ed6d877834ab686&mpshare=1&scene=23
&srcid=0102eq02kz2LSXe91k7ioKXi#rd](http://mp.weixin.qq.com/s?__biz=MzI1MTIzMzI2MA==&mid=2650561534&idx=1&sn=da508af331c81614b56fce4102c7958f&chksm=f1feed7dc689646b5d65f7d797612bdfdadf72c287ee2e92b4eb11310df2572945e217d2f729&mpshare=1&scene=23&srcid=0102eq02kz2LSXe91k7ioKXi#rd)

归并排序

[http://mp.weixin.qq.com/s?__biz=MzI1MTIzMzI2MA==&mid=2650561534&idx=1
&sn=da508af331c81614b56fce4102c7958f&chksm=f1feed7dc689646b5d65f7d797
612bdfdadf72c287ee2e92b4eb11310df2572945e217d2f729&mpshare=1&scene=23
&srcid=01027kbJ8V739HhagK3LadHe#rd](http://mp.weixin.qq.com/s?__biz=MzI1MTIzMzI2MA==&mid=2650561534&idx=1&sn=da508af331c81614b56fce4102c7958f&chksm=f1feed7dc689646b5d65f7d797612bdfdadf72c287ee2e92b4eb11310df2572945e217d2f729&mpshare=1&scene=23&srcid=01027kbJ8V739HhagK3LadHe#rd)

快速排序：

[http://mp.weixin.qq.com/s?__biz=MzI1MTIzMzI2MA==&mid=2650561538&idx=1
&sn=d2b1453e881e185b5f5b508bef74bf34&chksm=f1feea81c6896397c2dc15d586
d9fc0d31b840a06b43bf34297d98963662b92c7602bf30e607&mpshare=1&scene=23
&srcid=0102SofUyWTPAKfLDWEFRpWs#rd](http://mp.weixin.qq.com/s?__biz=MzI1MTIzMzI2MA==&mid=2650561538&idx=1&sn=d2b1453e881e185b5f5b508bef74bf34&chksm=f1feea81c6896397c2dc15d586d9fc0d31b840a06b43bf34297d98963662b92c7602bf30e607&mpshare=1&scene=23&srcid=0102SofUyWTPAKfLDWEFRpWs#rd)

堆排序：

[http://mp.weixin.qq.com/s?__biz=MzI1MTIzMzI2MA==&mid=2650561529&idx=1
&sn=48578c4815c33b9e9605b7e898bb948f&chksm=f1feed7ac689646c03e58ceabe
b2103691477146d8c1798c3b18dab75287f7bae2b890200905&mpshare=1&scene=23
&srcid=0102HojJB8oq4wTBYk7wmQrjP#rd](http://mp.weixin.qq.com/s?__biz=MzI1MTIzMzI2MA==&mid=2650561529&idx=1&sn=48578c4815c33b9e9605b7e898bb948f&chksm=f1feed7ac689646c03e58ceabe b2103691477146d8c1798c3b18dab75287f7bae2b890200905&mpshare=1&scene=23&srcid=0102HojJB8oq4wTBYk7wmQrjP#rd)

树形选择排序：

基数排序

一、基本概念

算法：规则的有限集合，求解特定问题的过程描述/操作步骤/指令序列

算法的特性：有穷性、确定性、可行性、输入、输出

算法设计的要求：正确性、可读性、健壮性、高效率、低存储

数据结构的形式定义：Data_Structure=(D, R)

D:数据对象 R:定义在 D 上的关系集

抽象数据类型 (ADT): ADT= (D, R, P) P:对 D 和 R 的基本操作集

算法复杂度：时间/空间复杂度 时间： $T(n)=O(f(n))$ 空间： $S(n)=O(f(n))$

二、表结构

线性结构概念：一个数据元素的有限序列

顺序存储：由位置描述逻辑关系

链式存储：由指针描述逻辑关系

1. 顺序表

定义：用一组**地址连续**的存储单元存储线性表的数据元素

特点：空间利用率高、随机存取、便于查找、

线性表的顺序存储结构

```
typedef struct {  
    Type *elem;    //Type: 用户定义数据类型  
    int n;    //线性表的长度(length)  
    int N;    //当前分配的存储容量(listsize)  
} SList;
```

基本操作：

创建（开辟空间、长度赋 0、初始化存储容量）、

插入（判断存储容量与长度的关系、将插入位置之后的元素后移、插入）、

删除（判断边界合法性、找到元素、其后元素前移、长度减一）、

查找特定条件元素（判断是否越界，while 查找满足条件的元素）、

输出、判断是否为空、销毁

算法

归并两个有序顺序表（创建新表，比较，置入）

2. 链表

定义：以结点序列表示的线性表

需要插入数据元素时，申请一个结点的地址单元，并且将该单元链入线性表中

特点：存储空间利用率低（需要指针结点），逻辑位置相邻、物理位置不一定相邻，插入和删除操作比较灵活，不便于查找

```
typedef struct Node  
{  
    Type data;    //数据域  
    struct Node *next;    //指针域  
}Node,*LinkedList;
```

线性链表（单链表）：链表的每个结点只包含一个指针域

头结点：附加在第一个数据元素之前的结点，该结点的数据域一般为空、指针域

存放第一个数据元素的地址

头指针：线性链表中第一个结点或头结点的存储地址，是访问链表的起始点

动态链表：采用动态地址分配方法建立的线性链表（指针型描述）

需要时则分配一个存储区域用于从存储 Node 结构类型的数据元素

** 申请一个 Node 空间 Node *p=(Node*)malloc(sizeof(Node))

操作要点：注意头指针在操作时的特殊性，不要丢失头指针，不要丢失后续链表段

算法

归并两个有序链表(将另一链表中的结点插入到合适的位置，销毁该链表头指针)

连接两个链表（选取长度较短的一个[同时对两个链表进行遍历，先到达尾部的长度较短]，查找其尾指针）

循环链表：尾结点的 next 是头结点

算法：

Josephus 问题：总共 n 个，第 k 个开始，每数到第 m 个就弹出（循环计数，直到弹出 n 个数为止）

双向链表：每个结点都有指向前驱和后继的两个指针

```
typedef struct DNode
{
    Type data;           //数据域
    struct DNode *prior; //指向前驱
    struct DNode *next;  //指向后继
} DNode, *DLinkedList;
```

3. 栈与队列

栈：后进先出的线性表（LIFO）

top 指向栈顶元素上一位，base 指向栈底；当 top=base 时，栈空

根据输入序列，判断不可能的输出序列

顺序栈：用顺序存储结构表示的栈

```
typedef struct
{
    Type *base; //栈底指针
    Type *top;  //栈顶指针
    int stacksize; //当前分配的存储容量
} SStack; //顺序栈结构类型
```

操作：创建（base=空间基址）、进栈（空间若不足则调整空间，对 base 和 top 重新赋值[地址改变]，存入数据）

栈链：用链式存储结构表示的栈

```
typedef struct SNode
{

```

```
Type data;    //数据域
struct SNode *next;    //指针域
} SNode, *LinkStack;
注意：栈是否为空，是否会发生溢出
```

队列：先进先出的线性表（FIFO）

顺序队列：用顺序存储结构表示的队列。

```
typedef struct
{
    Type *front;    //队首指针
    Type *rear;    //队尾指针
    int queuesize;    //当前分配的存储容量
} SQueue;    //顺序队列结构类型
```

假溢出问题：队列前部还有空间时，队列后已经无法入队了

链队列：用链式存储结构表示的队列。

```
typedef struct QNode
{
    Type data;    //数据域
    struct QNode *next;    //指针域
} QNode, *LinkQueue;    //链队列结构类型
```

循环队列：为了避免假溢出问题，为了便于判断设置队首指针在首元素的前一个位置或者队尾指针在尾元素的下一个位置

```
typedef struct
{
    Type elem[N];    //N: 元素最大个数
    int front;    //队首指针
    int rear;    //队尾指针
} SQueue;
```

空队列： $rear=front$ 满队列： $(rear+1)\%MAXSIZE=front$

队列中元素的个数： $(rear-front+MAXSIZE)\%MAXSIZE$

4. 栈/队列的应用

栈

进制转换（10 进制到 d 进制） $n=(n/d)*d + n\%d$ 将 $n\%d$ 的数据逆序输出就是转换后的数字

迷宫问题：太长了不会考

括号匹配检验：遇到左括号进栈，遇到右括号则看是否与栈顶元素匹配，若匹配则出栈，若不匹配则直接返回 false，若到最后栈非空也是返回 false

中缀转后缀：遇到数值则入操作数栈，遇到运算符则与运算符栈中的栈顶元素比较，若栈顶元素优先级大于等于该运算符，则依次出栈并进栈到操作数栈至小于该运算符优先级，遇到右括号则出栈至左括号（优先级设置为特殊）为止

手动转换→按照运算优先顺序添加括号，并将括号中的符号移动到相应括号后，

去掉括号即得到后缀表达式

表达式求值：在中缀转后缀的基础上，运算符出栈后取出操作数栈的两个栈顶元素进行运算，将运算结果入栈，最终栈中剩下的一个数值就是最后结果

队列

依次输出 n 个随机数中的奇数和偶数（间隔输出），直到其中一种输出完毕：将 n 个随机数中的奇数和偶数分别寄存在队列中

递归

特点：递归出口至少一个，经过有限次的递归调用后能够导致递归出口的出现
需要解决的问题（设计算法思路）：递归结束条件，递归返回值，递归输入的数据

汉诺塔问题：对三个圆盘的情况进行分析就可以了

背包问题：递归查找背包剩余的最小容积

5. 更多应用

数组：下标计算，位置计算等

十字链表：创建行列指针数组，用于存储稀疏矩阵等

三、树结构

1. 基础

树：一种非线性结构， n 个结点的有限集合

基本特点：递归性质、层次结构

空树：结点数为 0 的树

树有且只有一个根结点，其他任意结点有且仅有一个前驱，可有任意多个后继

孩子结点、子孙结点、兄弟结点、双亲结点、祖先结点。。

树的结点：包含一个数据元素及若干个指针

结点的度：拥有子树的个数

叶子结点：度为 0 的结点

分支节点：度大于 0 的结点

树的度：所有结点的最大度

结点的层次：根节点为第一层

树的高度/深度： $\max(\text{结点的层次})$

森林： m 棵互不相交的树的有限集

二叉树：结点的度 ≤ 2 的树

满二叉树：高度 k 且具有 $2^k - 1$ 个结点的二叉树

完全二叉树：前 $k-1$ 层为满二叉树，第 k 层结点都集中在左侧

二叉树的性质：

第 i 层最多有 2^{i-1} 个结点，

高度为 k 的二叉树最多有 $2^k - 1$ 个结点，

若二叉树的叶子结点数为 n_0 ，度为 2 的结点数为 n_2 ，则 $n_0 = n_2 + 1$ ，

具有 n 个结点的完全二叉树 T 的高度为 $\lfloor \log_2 n \rfloor + 1$
完全二叉树的性质：在有 n 个结点的树中，对于编号为 i 的结点，
双亲结点的编号是 $\lfloor i/2 \rfloor$ ($i > 1$) 左孩子编号是 $2i$, 右孩子编号是 $2i+1$
若 $2i \leq n$, i 为分支结点; $2i > n$, i 为叶子结点
若 n 为奇数，则所有结点都有左右孩子; 若 n 为偶数，则编号为 $n/2$ 的分支结点 (最大的分支结点，其后均为叶子结点) 只有左孩子

二叉树的顺序存储结构: 从根节点开始, 自上而下, 自左而右排成一个线性序列, 各结点对应的位置标号为其在完全二叉树中的标号 (适用于完全二叉树)

二叉树的链式存储结构: (二叉链表)

```
typedef struct Tnode
{
    Type data; //数据元素域
    struct Tnode *lc; //左孩子域
    struct Tnode *rc; //右孩子域
} Tnode, *Tree;
```

求二叉树的高度: 分别求左右子树的高度 ld, rd , 高度 $= \max(ld, rd) + 1$

2. 二叉树遍历

作用: 可以使二叉树线性化

二叉树的组成: 根节点 (D), 左子树 (L), 右子树 (R)

先序遍历: (PreOrder) 根 $\rightarrow$ 左 $\rightarrow$ 右

中序遍历: (InOrder) 左 $\rightarrow$ 根 $\rightarrow$ 右

后序遍历: (PostOrder) 左 $\rightarrow$ 右 $\rightarrow$ 根

非递归算法: 使用链栈, 向左孩子依次遍历并且将其入栈, 按照所需的顺序访问或出栈

```
typedef struct Tnode
{
    Type data;
    struct Tnode *lc;
    struct Tnode *rc;
} Tnode, *Tree;

typedef struct SNode
{
    Tree tnode; //数据域
    struct SNode *next; //指针域
} *Stack;
```

3. 特殊二叉树

线索二叉树: 在结点的空指针域中存放指向某次遍历得到的后继或前驱的指针

线索: 前驱线索, 后继线索

```
typedef struct Node
{
    Type data; //数据元素域
    struct Node *lc; //左孩子域
    struct Node *rc; //右孩子域
    int Tag; //0: 分支结点, 1: 叶子结点
}
```

```
} Tnode, *Tree;
```

哈夫曼树：最优二叉树，WPL 值最小（树不唯一）

路径：从一个结点到另一个结点之间的分支序列

路径长度：路径上的分支数

树的路径长度：从根节点到树中每一个叶子结点的路径长度之和

结点的权值

带权路径长度：从根节点到某个叶子结点之间的路径长度与该节点的权值乘积

树的带权路径长度：（WPL）树中所有叶子结点的带权路径长度之和

$WPL = \sum w_i l_i$, $i=1, \dots, n$

性质： $n_0 = n_2 + 1$ ，没有度为 1 的结点，故总结点数为 $2n_0 - 1$

构造算法：选取根节点权值最小的两棵子树（若有多个相等的则任取）作为左右子树，新的二叉树根节点权值等于这两个子树根节点的权值之和，将其左右子树从序列中去除，并将该结点置于序列的最后。最终得到的序列最后一个根节点即为哈夫曼树的根结点

注意：构造哈夫曼树时，没有规定左右子树的次序，一般按序列中的默认顺序就好了，权值越大的叶子结点离根结点越近

```
typedef struct
```

```
{ char data; //存放结点的元素值
  int Wt;    //存放结点的权值
  int Np, Nl, Nr; //指向双亲、孩子结点指针
```

```
} HTree;
```

```
HTree Ht[2*N]; //后 N 个位置用于存储过程
```

前缀编码：字符集中任何一个字符的编码都不是另一个字符编码的前缀

哈夫曼编码：带权总量最小的前缀编码

求哈夫曼编码：根据权值构造哈夫曼树，左分支为 0，右分支为 1，每个元素的哈夫曼编码即为根结点到当前结点（叶子结点）的路径代码。利用顺序存储结构来求的时候，需要标记各根结点的左右子树位置编号

4. 树的存储结构

双亲表示法：顺序表存储树的结点，每个结点指针域指向双亲位置

```
typedef struct Node
```

```
{ Type data; //数据元素
  int parent; //双亲位置
```

```
} TNode; //结点结构
```

```
typedef struct
```

```
{ TNode tree[MaxSize];
```

```
  Int nodenum; //结点数
```

```
} PTree; //树的存储结构
```

孩子表示法：每个节点建立一个链表，用于存储该节点的所有孩子结点的位置

```
typedef struct Node
```

```
{ int child;
  struct Node *next;
```

```

}*Tnode; //孩子链表结点结构
typedef struct
{
    Type data;
    Tnode firstchild;
} Tchild; //表头指针结构
typedef struct
{
    Tchild T[MaxSize];
    int r, n; //根的位置, 结点数
} CTree; //树的存储结构

```

孩子兄弟表示法：二叉树的存储结构表示树，第一个孩子为左孩子，双亲结点的兄弟依次存储为右孩子

```

typedef struct Tnode
{
    Type data; //数据元素域
    struct Tnode *Child1; //第1个孩子
    struct Tnode *Sibling; //下1个兄弟
} Node, *Tcs;

```

森林/树与二叉树的转换：孩子兄弟表示法

四、图结构

1. 基础

图：一种非线性数据结构，数据元素是顶点，数据对象是顶点集，主要研究两顶点之间的连通关系

子图：顶点集和边集都是原来的图的子集

顶点：(Vertex)

顶点集：(V) 研究的所有顶点的集合

关系集：(VR) 两个顶点之间关系的集合

弧：(Arc) $\langle v, w \rangle$ 表示从 v 到 w 的一条弧

弧头：(Head) 箭头端顶点

弧尾：(Tail) 非箭头端顶点

边：(Edge) (v, w) 表示既有 $\langle v, w \rangle$ 又有 $\langle w, v \rangle$

m ：图中边/弧的数目， n ：图中顶点的数目

完全图： $m=n(n-1)/2$ 的无向图

有向完全图： $m=n(n-1)$ 的有向图

稀疏图： $m \ll n(n-1)$ 的图

稠密图： m 接近于 $n(n-1)/2$ 的图

权值：标示边或者弧某种特性的参数值

网：(有向/无向) 带权图

无向图：由顶点集和边集构成的图

邻接点：由边连接的两个点互为邻接点

顶点的度： $(TD(v))$ 和该顶点相邻接的边的数目

边数等于度数的一半

有向图：由顶点集和弧集构成的图

邻接点：对于 $\langle v, w \rangle$ v 邻接到 w ， w 邻接到 v

入度： $ID(v)$ 以该顶点为弧头的弧的数目

出度： $OD(v)$ 以该顶点为弧尾的弧的数目

顶点的度： $TD(v) = ID(v) + OD(v)$

边数=出度和=入度和

路径长度：路径上边/弧的数目

回路（环）：第一个顶点和最后一个顶点相同的一条路径

简单路径/回路：不重复经过同一顶点

v 和 w 连通： v 和 w 之间有路径

连通图：无向图中任意两个顶点都连通

连通分量：无向图中的极大连通子图

强连通图：有向图中任意两个顶点都相互可达

强连通分量：有向图中的极大强连通子图

有向树：只有一个顶点的入度为 0，其他顶点的入度都为 1 的有向图

生成森林：含有图中全部 n 个顶点的 m 棵互不相交的有向树

存储方式

邻接矩阵：数组表示法

邻接表：链式存储结构；每个顶点对应一个结点，由所有顶点结点构成一个顺序表，每个顶点建立一条链，该链由它所有邻接点构成

```
typedef struct Anode
{
    int Vi; //邻接点在顺序表中的位置
    int Wi; //权值(用于网。图可省略)
    struct Anode *next; //指向下一个邻接点
} ALink; //邻接表存储结构

typedef struct
{
    Type data; //顶点元素值
    ALink *Vh; //邻接点链表头指针
} Vnode; //表头指针结构

typedef struct
{
    Vnode V0[N]; //链表头顺序表
    int n, m; //实际顶点数，边或弧数
    int V1[N]; //访问标示
} AList; //图的邻接表存储结构

逆邻接表：对于有向图
```

遍历

DFS：适用于找一条路径

从图中某个顶点 v 出发，访问该顶点，查找 v 第一个未被访问的邻接点 v_1 ，访问

该顶点，若当前顶点没有未访问邻接点，则返回上一层查找，直到图中没有未被访问顶点为止

BFS：适用于找最优路径

从图中某顶点 v 出发，访问顶点 v ，依次访问 v 的所有未被访问的邻接点 v_k ，按照 v_k 被访问的次序依次访问未被访问过的邻接点，直到图中没有未访问顶点为止

操作：判断两顶点是否连通

2. 最小生成树

生成树：含有图 G 的所有 n 个顶点、且有 $n-1$ 条边的连通图

最小生成树：在一个连通网的所有生成树中，代价之和最小的生成树

性质：对于图 G 顶点集 V 的非空子集 U ，具有最小权值的边 (u, v) ， $u \in U$ ， $v \in V-U$ ，则存在一棵包含边 (u, v) 的最小生成树（任意顶点邻边中权值最小的边必定在最小生成树中）

Prim 算法（加点法） $O(n^2)$ 稠密图

任取一个顶点 v 作为生成树的根，然后往生成树上添加新顶点 u ，顶点 u 满足其与生成树上某个顶点连通，并且在所有与生成树连通的边中权值最小，不断往生成树上添加顶点至生成树包含所有顶点为止。

实现：设置辅助数组（或设置标记），用于记录 $V-U$ 集中的每个顶点以及 U 集中所有顶点代价最小的边，查找满足条件的顶点 k ，将该顶点 k 从 U 集移动到 $V-U$ 集中，更新 U 集中所有顶点的最小边值（即根据 k 的邻边权值进行更新）

辅助数组的数据结构：

```
typedef struct
{
    int Vi; //顶点序号
    int Wi; //邻接边的最小权值
} Aedge;
```

Kruskal 算法（加边法） $O(mn)$ 稀疏图

构造一个只含 n 个顶点，边集为空的子图 S ，从权值最小的边开始添加，若新加边不会在 S 中产生回路，则在 S 上加上这条边，不断往图中加边至有 $n-1$ 条边为止。

实现：先对所有边根据权值进行排序（选用堆排序·大顶堆）选取最小边，若不成环则加入

成环的判断：可以用标记法来判断，或寻找连通分支尾部，判断是否相同（相同则成环）

//定义边集数组类型

```
typedef struct
{
    int v1; //起始顶点
    int v2; //终端顶点
    int Wi; //权值
} edge;
```

3. 最短路径

求简单路径：从开始顶点进行 DFS 到终点

最短路径：路径上各边权值之和最小

单源最短路径问题：设顶点 v 是源点，要求在有向网中找到从源点 v 到其他各顶点的最短路径

Dijkstra 算法 $O(n^2)$

把图中所有顶点分成两组，第一组 U 包含已确定最短路径的顶点，初始值只含源点 v ，第二组 $V-U$ 包含上位确定最短路径的顶点，初始值含有图中除源点 v 之外的其他顶点。按路径长度递增的顺序计算源点 u 到其他各顶点的最短路径，逐个把第二组中的顶点加到第一组中去，直至 $U=V$ 。

实现：初始化到各顶点的最短路径（与源点邻接的顶点为其邻接边的权值，其余的顶点为 0 或者无穷，视情况而定），选取当前最短路径值最小的一个顶点加入集合 U ，对每个新加入的顶点，重新计算其邻接顶点的最短路径，反复比较直至 $U=V$

```
typedef struct
{
    int n;        //顶点数
    char Ve[N+1]; //顶点元素值
    int Vr[N][N]; //邻接矩阵
    int Wi[N];     //当前最短路径
    int Vi[N];     //最短路径序列(链)，保存上一个顶点
    int vt[N];     //访问标识(0 或 1)
} Amatrix; //当前算法邻接矩阵存储结构
输出最短路径可用栈实现，或者使用辅助数组逆序输出
```

Floyd 算法 $O(n^3)$ 不考

从 v_i 到 v_j 的所有可能存在的路径中，选出一条加权长度最短的路径

```
typedef struct
{
    int n;        //顶点数
    char Ve[N+1]; //顶点元素值
    int Vr[N][N]; //邻接矩阵
    int Wi[N][N]; //当前最短路径长度
    int path[N][N][N]; //当前最短路径 (N 条从某点到另一点的路径经过的
                        //顶点)
} Amatrix; //当前算法邻接矩阵存储结构
```

4. 关键路径

有向无环图：一个没有回路的有向图

检查是否成环：标记法、深度优先搜索看是否遇到回边、拓扑排序后所有的顶点若都在拓扑序列中则无环

拓扑排序 $O(n+m)$

从有向图中随机选取一个入度为 0 的顶点输出，上去该顶点以及所有以它为尾的弧，重复操作至图空或无入度为 0 的顶点为止

实现：求各顶点的入度，将入度为 0 的顶点入队，当队列非空是，输出队首元素，将该顶点的所有邻接点入度减 1，若有新地入度为 0 的顶点则将该顶点入队，重复操作至图空或无入度为 0 的顶点为止

```
typedef struct Anode
{
    int Vi; //邻接点序号
    int Wi; //权值
    struct Anode *next; //下一个邻接点
} Anode; //链表存储结构
typedef struct
{
    Anode *Vh; //链表头指针
    char data; //顶点元素值
    int VI; //逆邻接值（保存上一个顶点）
    int WI; //当前关键路径值
    int ID; //顶点的当前入度
} Vnode; //表头存储结构
typedef struct
{
    int n; //顶点数
    Vnode V[N]; //图存储结构
    int VO[N]; //入度=0 的顶点队列
} Alist; //邻接表存储结构
```

AOE 网：用顶点表示子工程，用弧表示活动（或优先关系）的有向图（网）

源点：入度为 0 的顶点（始点）

汇点：出度为 0 的顶点（终点）

关键路径：从源点到汇点的最长路径

关键顶点：关键路径上的顶点

算法 $O(n+m)$

计算 ID[]，将 ID=0 的顶点入队，若队列非空，则{队首出队；更新顶点入度和顶点队列；更新关键路径值，更新逆邻接值}，从汇点出发，沿逆邻接指针找到关键路径并输出

五、查找

1. 基础

查找：在数据表中查找满足给定值的第一个记录或全部记录

查找成功：找到满足条件的记录，返回结果所在记录的指针

查找失败：未能找到满足条件的记录，返回空指针

关键字：记录中某个成员数据项的值，用以识别一个记录

主关键字：可以唯一识别记录的关键字

次关键字：可以识别若干个记录的关键字

ASL(平均查找长度): $ASL = \sum P_i C_i$ P_i 为查找数据表中第 i 个记录的概率, $\sum P_i = 1$, C_i 为找到第 i 个记录时已比较的次数

2. 静态查找

定义：没有改变数据表的查找

顺序查找 $O(n)$ $ASL = n/2 + 1$

依据顺序查找的方法实现对数据表的查找算法，从数据表中的最后一个记录（或第一个记录）开始，逐个比较关键字和给定值，并判断查找是否成功

实现：顺序查找记录的关键字，若找到则返回该记录在数据表中的位置，否则返回 0

折半（二分）查找 $O(\log n)$ $ASL = ((n+1)\log_2(n+1))/n - 1 \approx \log_2(n+1) - 1$

对有序表（表中记录按照关键字值有序排列）的查找算法

实现：设置区间左右边界以及中点（左右边界的中点，取下界），目标关键字与中点关键字比较，调整上/下边界为中点左/右边的点，重复以上过程直至上下边界重合并且不为关键字或找到关键字

分块查找

将数据按照一定原则存入数据表，使数据表划分为若干个逻辑子表，逻辑子表由索引表（由每个逻辑子表的~~最大关键字项~~、子表首元素位置、子表元素个数构成的一个有序表）划分

实现：先用顺序查找法在索引表（存储逻辑子表信息的结构）中查找索引记录（目标关键字小于索引记录），再依据该记录值在数据表（实际存储数据的位置）中的相应逻辑子表查找所需要的记录

```
typedef struct
{
    KeyType index; //最大关键字项
    int start; //子表首元素位置
    int length; //子表元素个数
} IndexList; //逻辑子表结构
IndexList LI[M];
```

3. 动态查找

定义：可能需要在数据表中进行插入操作或删除操作的查找

中序遍历二叉排序树可以得到排序序列

二叉排序树 ASL 根据形态确定，同一数量不同形态的值不一定相同

定义：一棵空树或者是满足下列关系的一棵二叉树（左小右大）

```
{
    左子树上的所有结点值均小于根结点值
    右子树上的所有结点值均大于或等于根结点值
    左右子树分别是二叉排序树
}
```

实现：

（仅查找）递归实现，若找到叶子节点的孩子则返回 false，若找到目标结点则

查找成功, 否则根据当前结点与目标结点的大小比较决定往左还是往右向下查找 (插入) 在查找过程中标记上一位置的头结点, 若查找成功, 返回 0 (不进行插入操作), 否则, 插入值为 key 的结点并返回 1

```
typedef struct Tnode
{
    KeyType key; //关键字
    ElemType *otherinfo;
    struct Tnode *lc; //左指针
    struct Tnode *rc; //右指针
} Tnode, *Tree; //二叉排序树的存储结构
```

操作:

删除节点 (被删除结点考虑三种情况: 叶子结点、只有一个子树 (左或右) {重接子树 (右或左)}、既有左子树又有右子树 {将左子树的最右子节点作为新的根节点, 若最右子节点存在左子树, 则将其重接到最右子节点的前驱, 作为其新的右子树})

平衡二叉树 (AVL 树) $O(\log n)$

定义: 一棵空树或者是满足下列关系的一棵二叉树

```
{
    左右子树都是平衡二叉树
    左右子树的深度之差  $\leq 1$ 
}
```

平衡因子 = 左子树的深度 - 右子树的深度

平衡二叉树的平衡因子: (-1、0、1)

二叉树上所有节点的平衡因子的绝对值都小于等于 1, 则他是一棵平衡二叉树

构造平衡二叉树方法: 在构造二叉排序树的过程中, 若出现不平衡的结点, 需采用平衡方法将二叉树调整成平衡二叉树

```
typedef struct Tnode
{
    KeyType key; //ElemType *otherinfo
    int bf; //结点的平衡因子
    struct Tnode *lc; //左孩子指针
    struct Tnode *rc; //右孩子指针
    struct Tnode *parents; //双亲指针
} *Tree; //平衡二叉树类型定义
```

操作:

调整平衡二叉树 (要点: 将**关键字中间值**的结点作为根节点, 再重新连接相关结点, 构成平衡二叉树)

4. 哈希

散列表 (Hash List): 以数据元素的关键字 key 作为自变量构造一个哈希函数 Hash(key), 并将函数值定义为该数据元素的存储地址。为数据元素在列表中的位置和它的关键字值建立一个确定的关系

哈希地址: 哈希函数的值

构造方法

直接定址法（取关键字的线性函数作为哈希函数 $H(key)=a*key+b$ ）：地址集合的大小 $\geq$ 关键字集合的大小

除留余数法（设哈希表表长为 m ，取哈希函数为 $H(key)=key\%p$, $p \leq m$ ）：要选择合适的 p 值，尽可能减小冲突，一般取 p 值为素数

冲突处理（为产生冲突的哈希地址寻找下一个哈希地址）

开放定址法（ $H_0=H(key)$ $H_i=(H_0+d_i)\%m$ m 为表长）：

线性探测再散列： $d_i=c*i$

二次探测再散列： $d_i=1^2, -1^2, 2^2, -2^2, \dots$

链定址法（将所有哈希地址相同的数据元素都存放在同一个链表中）：平均查找长度较短，适合于构造表前无法确定表长的情况

实现：若 $Hash[H_i]=NULL$ 则查找失败，否则向下搜索直到找到目标或 $Hash[H_i]=NULL$ 为止

```
typedef struct Lnode
{
    KeyType key; //ElemType data
    LNode *next;
} *Chain; //链地址法结构
Chain HashL[m]; //m 为表长
```

ASL：由处理冲突方法和装填因子决定

装填因子 a =哈希表中实际填入的记录数/哈希表的长度 a 越大越容易发生冲突

线性探测再散列： $ASL=(1/2)(1+(1/(1-a)))$

六、排序

1. 基础

稳定性：若 $r_i=r_j$ 且 $i < j$ 时，排序后 r_i 仍然领先于 r_j ，则称排序为稳定的

分类

物理排序：根据关键字值在数据表中重新排列记录的存储位置

索引排序：根据关键字值建立索引表，在输出数据表的记录时，由索引表描述输出顺序

比较式排序：根据数据表中两个元素值的大小，确定这两个元素的存储位置

基本操作：比较两个关键字的大小、交换两个记录的位置

数据区：有序区、无序区

计算式排序：根据元素值，计算它在数据表中的存储位置

2. 插入排序 $O(n^2)$

在有序区找第 i 个元素的插入位置

实现：设置哨兵保存待插入元素，将有序区中排在该元素以后的元素进行后移，将待插入元素插入到目标位置

3. 希尔排序 一趟为 $O(n^2/m)$

分逻辑区的插入排序

实现：将数据表分成 m (m 同时为增量) 个子序列，对每个子序列分别进行插入排序，逐步减少子序列数 m ，直至 $m=1$ 为止

4. 选择排序 $O(n^2)$

在无序区找第 i 小值的元素

实现：设置初始索引值为无序区第一个元素的索引，在无序区中进行比较，遍历无序区，若此时索引值与初始索引值不同，则交换元素位置

5. 冒泡排序 $O(n^2)$

将无序区的最大值元素交换到最后

实现：每次循环从第一个元素开始，遇到满足条件则交换，直到一趟没有交换记录时就可以 break

6. 归并排序 $O(n\log_2 n)$

空间 $O(n)$

将两个或两个以上的有序子序列合并为一个有序序列

实现：将待排序表分为两部分，分别对两部分归并排序，再将这两个部分进行归并，归并过程使用辅助数组存储结果，最终再将辅助数组中的数据复制到子序列中

7. 快速排序 $O(n\log_2 n)$ 最好为 $O(n)$ 最差为 $O(n^2)$ 一趟快速排序为 $O(n)$

空间 $O(\log_2 n)$

通过一趟快速排序将待排序数据分割成独立的左中右三个部分，其中，中部分支行一个数，左边部分所有关键字都比这个数小，右边部分的所有关键字都比这个数大，然后再对这两部分记录进行快速排序，知道所有记录整理成有序序列

一趟快速排序：

实现：每次使用一趟快速排序划分序列，直到所有的子序列都只含一个元素为止
一趟快速排序划分序列，使用子序列左右端元素索引，对子序列最左端元素设置哨兵，当左索引小于右索引时{从最右侧开始，若元素大于等于哨兵，则将右索引值减小直到元素小于哨兵时；修改左索引对应元素为右索引对应元素的值；从最左侧开始，若元素小于等于哨兵，则将左索引增加直到元素大于哨兵时；修改右索引对应元素为左索引对应元素的值}，修改左索引对应元素的值为哨兵的值，返回枢轴位置

改进：划分前，对枢轴记录进行预处理，取子序列左、右、中位置三个元素中间大小的为枢轴记录，对应算法也需要改进

8. 堆排序 $O(n\log_2 n)$

堆：完全二叉树

小顶堆：左右子结点均大于等于根结点

大顶堆：左右子结点均小于等于根结点

实现：建堆以后，将根节点与最后一个结点交换位置，然后调整交换后的堆（此时原来堆中最后一个结点不包括在堆中了），循环上述步骤直至堆中只有一个结

点为止

操作:

调整堆: 自上而下调整关键字的位置, 直至满足小/大顶堆的特征

调整堆实现: $O(\log_2 n)$ (HeapAdjust) 以小顶堆为例, 选取子结点中较小的一个与该节点比较, 若该结点值大于子结点则交换, 选取该子节点作为根节点循环上述步骤直至根节点值小于选出的子节点值为止或根节点为叶子结点为止

建堆: 选取最后一个分支节点 ($\lfloor n/2 \rfloor$) 开始调整, 逐个向前选取结点, 直到调整完根节点为止

9. 树形选择排序 (锦标赛排序) $O(h*n)$

一种基于完全二叉树的排序算法。首先对 n 个记录的关键字进行两两比较, 然后在 $n/2$ 个较小/大者之间再进行两两比较, 如此重复, 直至选出最小的记录为止

实现: 计算满二叉树的高度 ($h = \lceil \log_2 n \rceil$), 将满二叉树 (总共 $2^h - 1$ 个结点) 的结点值都置为 ∞ (或 $-\infty$), 将 n 个值存在完全二叉树的底层, 构建完全二叉树 ($O(h*n)$) {根节点等于子节点中较小/大的一个}, 将树根节点的值取出并将所有等于该值的结点置为 ∞ (或 $-\infty$), 直到取出所有结点为止

10. 基数排序 $O(k(n+r))$ r 为基数 (10), k 为“分配-收集”趟数 (关键字位数) 空间 $O(n+r)$

一般采用链表存储结构, 将单关键字看成由多个数位 (或字符) 构成的多关键字, 并采用“分配-收集” (使用排序一次以后的序列进行下一次排序) 的方法进行排序, 从个位数开始进行排序比较

实现: 将待排序记录存在链表中, 开辟 r 个链队列, 分配时将当前关键字位值相同的记录分配到同一个链队列中, 收集时按当前关键字位取值从小到大 (或从大到小) 顺序将各链队列链成一个链表, 重复以上步骤直至最高位

```
typedef struct Node
{
    int data; //数据域
    int data1; //辅助变量, = (data/10位数-1) %10
    struct Node *next; //指针域
} Node;

typedef struct
{
    Node H[Radix]; //表头指针 Radix 为基数
    Node R[Radix]; //链尾指针
} Head; //链表存储结构
```

11. 稳定性

稳定算法: 插入排序、冒泡排序、归并排序、基数排序

不稳定算法: 希尔排序、选择排序、堆排序、快速排序